

Problem Set 3

E. Points

TIANZHEYU SETTINGS LISTS BLOG TEAMS SUBMISSIONS CONTESTS

☒ show unofficial

tianzheyu submissions

tianzheyu submissions							
#	When	Who	Problem	Lang	Verdict	Time	Memory
379309992	Jun/18/2026 16:49UTC+10	tianzheyu	D - Points	C++23 (GCC 14-64, msys2)	Accepted	1062 ms	75800 KB

Process

The question is saying that we have a Cartesian coordinate system where (0,0) is located in the bottom-left corner. We have 3 operations:

1. add: we add point (x, y) into our coordinate system
2. remove: remove point (x, y) from our coordinate system
3. find: we return the point strictly above and strictly to the right of point, we will pick the leftmost one, if it is not unique, we will choose the bottommost one.

Challenges and Overcoming

The naive solution is $O(n^2)$ which is $4 * 10^{10}$ which will have time limit exceeded. After getting some hints, I know we can consider the coordinate system as a range tree. We can make x values of all points that appeared in the queries as the base range of our segment tree. However, since the maximum x-coordinate can be up to 10^9 , we cannot build a segment tree of that size directly.

1. Coordinate Compression

Since there are n queries, there are at most n unique x-coordinates. I collected all x-coordinates from the queries, sorted them, and removed duplicates using the classic `x.erase(unique(x.begin(), x.end()), x.end())`. This maps x-coordinates into a 1-based index range `[1, M]`, which can perfectly be the segment tree's range.

2. Segment Tree Design

Each node in the segment tree maintains the maximum y-coordinate within its mapped x-range. For the leaf nodes (which represent a specific unique x-coordinate), I used a set to store all the y-coordinates that share this exact x-coordinate. Whenever we want to add a point or remove a point, we need to insert or erase the value into the set in the leaf node. If there is no element left in the set, we will make maximum y-coordinate -1. And we will update along the path from that leaf node to the root.

3. Find Operation

We need to find a point strictly above and to the right. Instead of querying a full range and iterating, we will check if the current node's `max_y <= target_y` (there is no point strictly above the current node) or its `range.second <= target_x_index` (the range this node covers is less than `target_x_index`, i.e. all the nodes in that range are on the left side of the current point). This means no valid point exists in this subtree. We can

immediately return `{-1, -1}` in $O(1)$ time.

Apart from that, to satisfy the "leftmost" requirement, we always recursively search the left child first. If the left child search returns `(-1, -1)`, then we will check the right child.